

CHRONOS - Trình Quản Lý Nhiệm Vụ & Thời Hạn Cá Nhân

Bài tập lớn môn Lập trình Hướng đối tượng (OOP) - C++ Ngôn ngữ: C++ (sử dụng STL: `std::stack`, `std::vector`, `std::string` ...) Hình thức: Ứng dụng CLI (Command Line Interface)

1. Tổng Quan Dự Án

Chronos là một công cụ dòng lệnh (CLI) giúp người dùng quản lý nhiệm vụ, theo dõi thời hạn và mức độ ưu tiên. Hệ thống tính toán **điểm "Khẩn cấp" (Urgency Score)** dựa trên độ ưu tiên và thời gian còn lại đến deadline, đồng thời cung cấp tính năng **Hoàn tác (Undo)** thông qua cấu trúc `std::stack`.

Mục tiêu chính

- Theo dõi nhiệm vụ, thời hạn và mức độ ưu tiên.
- Tính toán **điểm Khẩn cấp** một cách thông minh.
- Quản lý nhiều nhiệm vụ trong các **Dự án (Project)**.
- Hỗ trợ **Hoàn tác** thao tác bằng `std::stack`.
- Xử lý các lỗi đặc biệt: **Nghịch lý thời gian & Tràn bộ đệm**.

2. Phân Chia Module Lớn

STT	Module	Mô tả ngắn
M1	Core Domain	Lớp <code>Task</code> , <code>Project</code> , các thuộc tính, getter/setter, nạp chồng toán tử
M2	Logic & Tính toán	Tính điểm Khẩn cấp, sắp xếp, validate dữ liệu, xử lý ngoại lệ
M3	Persistence & Undo	Lưu/đọc file, hệ thống Hoàn tác bằng <code>std::stack</code>

3. Chi Tiết Tính Năng Theo Module

MODULE 1: CORE DOMAIN (Lớp & OOP cơ bản)

1.1. Lớp `Task` (Nhiệm vụ)

- **Thuộc tính:**

- `id` (int) – Mã định danh duy nhất
- `title` (string) – Tên nhiệm vụ
- `description` (string) – Mô tả chi tiết
- `startDate` (Date) – Ngày bắt đầu
- `dueDate` (Date) – Ngày hết hạn
- `priority` (enum: LOW, MEDIUM, HIGH, CRITICAL) – Mức độ ưu tiên
- `status` (enum: PENDING, IN_PROGRESS, DONE) – Trạng thái

- **Phương thức:**

- Constructor (mặc định, tham số, copy)
- Destructor
- Getters / Setters
- `display()` – In thông tin nhiệm vụ ra màn hình

1.2. Lớp `Project` (Dự án)

- **Thuộc tính:**

- `projectId`, `projectName`, `description`
- `vector<Task> tasks` – Danh sách nhiệm vụ

- **Phương thức:**

- `addTask(Task)`, `removeTask(int id)`

- `getTaskById(int id)`
- `getAllTasks()` – Trả về danh sách
- `displayProject()` – Hiển thị toàn bộ dự án

1.3. Lớp `Date` (Hỗ trợ)

- Lớp riêng để biểu diễn ngày tháng (day, month, year).
- So sánh hai ngày, tính khoảng cách giữa hai ngày.

1.4. Nạp chồng toán tử (Operator Overloading)

- `operator<` – So sánh 2 nhiệm vụ theo **độ ưu tiên** rồi đến **ngày deadline**.
- `operator++` – **Gia hạn** thời hạn thêm 1 ngày (cả tiền tố `++task` và hậu tố `task++`).
- `operator==` – So sánh 2 nhiệm vụ có giống nhau không.
- `operator<<` – In thông tin task ra `ostream`.

MODULE 2: LOGIC & TÍNH TOÁN

2.1. Tính điểm Khẩn cấp (Urgency Score)

- Công thức gợi ý:

$$\text{Urgency} = (\text{Priority} * 10) + (100 / (\text{daysLeft} + 1))$$

- Càng gần deadline & độ ưu tiên càng cao → điểm khẩn cấp càng lớn.
- Sắp xếp danh sách task theo điểm khẩn cấp giảm dần.

2.2. Sắp xếp nhiệm vụ

- Dùng `std::sort` kết hợp `operator<` đã nạp chồng.
- Hỗ trợ sắp xếp theo: ưu tiên, deadline, điểm khẩn cấp.

2.3. Validate dữ liệu & Xử lý ngoại lệ

-  **Nghịch lý thời gian (Time Paradox):**

- Khi `dueDate < startDate` → ném `std::invalid_argument` hoặc class lỗi tự định nghĩa `TimeParadoxException`.
- In thông báo: *"Lỗi: Nghịch lý thời gian! Ngày hết hạn không thể trước ngày bắt đầu."*
- **✗ Tràn bộ đệm (Buffer Overflow Logic):**
 - Giới hạn số task trong project (VD: `MAX_TASKS = 9999`).
 - Khi user cố thêm 10.000 task trong vòng lặp → ném `BufferOverflowException`.
 - In thông báo: *"Lỗi: Vượt quá giới hạn lưu trữ! Số nhiệm vụ tối đa là 9999."*
- Validate input chuỗi, số, ngày tháng.

2.4. Tìm kiếm & Lọc

- Tìm kiếm nhiệm vụ theo tên/từ khóa.
- Lọc theo priority, status, ngày deadline.

MODULE 3: PERSISTENCE & UNDO

3.1. Hệ thống Hoàn tác (Undo Stack)

- Sử dụng `std::stack<ProjectState>` để lưu các trạng thái trước đó.
- **Trước mỗi thao tác** thay đổi (add/delete/update/extend) → push trạng thái hiện tại vào stack.
- **Lệnh undo** : Lấy trạng thái trên đỉnh stack ra → khôi phục.
- Giới hạn số lần undo (VD: 50 thao tác gần nhất).

3.2. Lưu trữ dữ liệu (File I/O)

- Lưu danh sách project & task vào file `.txt` hoặc `.csv` / `.json`.
- Tải lại khi mở chương trình.
- Hàm `saveToFile()` , `loadFromFile()` .

3.3. Quản lý phiên làm việc

- Tự động lưu mỗi khi thoát chương trình.

- Cảnh báo khi thoát mà chưa lưu.
-

MODULE 4: CLI & TRẢI NGHIỆM NGƯỜI DÙNG

4.1. Giao diện dòng lệnh (CLI Menu)

- Menu chính:

```
=== CHRONOS - Task Manager ===
1. Tạo dự án mới
2. Thêm nhiệm vụ
3. Xem danh sách nhiệm vụ
4. Sắp xếp theo khẩn cấp
5. Gia hạn deadline (++)
6. Hoàn tác (Undo)
7. Lưu / Tải dữ liệu
8. Thoát
```

- Nhập lệnh bằng số hoặc từ khóa (`add` , `list` , `undo` ...).

4.2. Hiển thị đẹp mắt

- In bảng nhiệm vụ với cột rõ ràng (dùng `iomanip`).
- Tô màu (nếu hỗ trợ): khẩn cấp = đỏ, hoàn thành = xanh.

4.3. Demo các trường hợp lỗi (cho video báo cáo)

- Minh họa **Nghịch lý thời gian**: nhập ngày hết hạn < ngày bắt đầu.
- Minh họa **Tràn bộ đệm**: vòng lặp `for` thêm 10.000 task.
- Minh họa **Undo** sau khi xóa nhiệm vụ.

4.4. Tài liệu & Kiểm thử

- File `README.md` hướng dẫn build & sử dụng.
 - Test case: thêm/xóa/sửa task, sắp xếp, undo, lỗi.
 - Báo cáo & video demo.
-

4. Phân Công 4 Thành Viên

Thành viên 1 - Trưởng nhóm (Module 1: Core Domain)

Vai trò: Kiến trúc sư hệ thống, chịu trách nhiệm phần “xương sống” của chương trình.

Nhiệm vụ cụ thể:

- ✓ Xây dựng lớp `Date` (hỗ trợ tính toán ngày tháng).
- ✓ Xây dựng lớp `Task` đầy đủ thuộc tính & phương thức.
- ✓ Xây dựng lớp `Project` quản lý danh sách task.
- ✓ Nạp chồng các toán tử: `<`, `++` (cả tiền tố & hậu tố), `==`, `<<`.
- ✓ Viết unit test cơ bản cho các lớp.
- ✓ Đảm bảo code tuân thủ chuẩn OOP (encapsulation, constructor đầy đủ).
- 🤝 **Phối hợp:** Thiết kế interface để TV2 và TV3 dùng.

Sản phẩm bàn giao: `Date.h/cpp`, `Task.h/cpp`, `Project.h/cpp`

Thành viên 2 (Module 2: Logic & Tính toán)

Vai trò: Chuyên gia thuật toán & xử lý lỗi.

Nhiệm vụ cụ thể:

- ✓ Cài đặt hàm tính **điểm Khẩn cấp (Urgency Score)**.
- ✓ Cài đặt thuật toán sắp xếp (`std::sort` + comparator).
- ✓ Cài đặt tìm kiếm & lọc nhiệm vụ.
- ✓ Xây dựng class **Exception** tự định nghĩa:
 - `TimeParadoxException`
 - `BufferOverflowException`
- ✓ Xử lý validate dữ liệu input.
- ✓ Viết test case cho 2 lỗi trên (chuẩn bị cho video demo).
- 🤝 **Phối hợp:** Dùng lớp `Task`, `Project` của TV1; bàn giao logic cho TV4 gọi từ menu.

Sản phẩm bàn giao: `UrgencyCalculator.h/cpp` , `Exceptions.h` , `TaskFilter.h/cpp`

Thành viên 3 (Module 3: Persistence & Undo)

Vai trò: Chuyên gia lưu trữ & quản lý trạng thái.

Nhiệm vụ cụ thể:

-  Cài đặt hệ thống **Undo** sử dụng `std::stack<ProjectState>`.
-  Định nghĩa cấu trúc `ProjectState` để lưu snapshot.
-  Cài đặt hàm `saveToFile()` & `loadFromFile()` (file `.txt` hoặc `.csv`).
-  Auto-save khi thoát chương trình.
-  Quản lý giới hạn số lần Undo (VD: 50).
-  Test case: thêm task → undo, xóa task → undo, gia hạn → undo.
-  **Phối hợp:** Dùng `Project` của TV1; tích hợp với menu của TV4.

Sản phẩm bàn giao: `UndoManager.h/cpp` , `FileHandler.h/cpp`

Thành viên 4 (Module 4: CLI & Trải nghiệm)

Vai trò: Frontend CLI, Tester & người làm tài liệu/video.

Nhiệm vụ cụ thể:

-  Xây dựng **giao diện CLI** (menu chính, sub-menu).
-  Tích hợp tất cả module của TV1, TV2, TV3 vào hàm `main()`.
-  In bảng task đẹp (dùng `iomanip` , căn lề).
-  Viết file `README.md` hướng dẫn build & sử dụng.
-  Soạn **kịch bản video demo** thể hiện:
 - Tạo project, thêm task.
 - Sắp xếp theo khẩn cấp.
 - Gia hạn deadline (++).
 - Hoàn tác (Undo).

- **Lỗi Nghịch lý thời gian.**
- **Lỗi Tràn bộ đệm (vòng lặp 10.000 task).**
-  Quay & dựng **video demo cuối cùng.**
-  **Phối hợp:** Là người tổng hợp code cuối cùng từ 3 thành viên còn lại.

Sản phẩm bàn giao: `main.cpp` , `Menu.h/cpp` , `README.md` , video demo, slide báo cáo.

5. Lộ Trình Đề Xuất (4 tuần)

Tuần	Công việc	Người thực hiện
Tuần 1	Thiết kế class diagram, chia interface	Cả nhóm họp; TV1 chốt thiết kế
Tuần 2	TV1 hoàn thành Module 1; TV2, TV3 bắt đầu	TV1, TV2, TV3
Tuần 3	TV2 + TV3 hoàn thiện; TV4 bắt đầu CLI	TV2, TV3, TV4
Tuần 4	Tích hợp, debug, viết báo cáo, quay video	Cả nhóm

6. Công Cụ Đề Xuất

- **IDE:** Visual Studio 2022 / VS Code + g++ / CLion
- **Quản lý mã nguồn:** Git + GitHub (mỗi thành viên 1 branch riêng)
- **Quay video:** OBS Studio
- **Vẽ class diagram:** draw.io / PlantUML

7. Tiêu Chí Đánh Giá

- Đầy đủ các tính năng OOP yêu cầu (class, operator overloading).
- Tính năng Undo hoạt động đúng với `std::stack`.
- Demo được 2 trường hợp lỗi đặc biệt.
- Code sạch, có comment, chia file rõ ràng.

- Báo cáo + video demo đầy đủ.
- Phân công nhóm công bằng & rõ ràng.



Chúc nhóm làm bài thành công! Hãy nhớ commit code thường xuyên và họp nhóm hàng tuần.